Solving Least Squares Problems with Gradient Descent: the Least Mean Square Algorithm

Copyright © Barry Van Veen 2016

AllSignalProcessing.com



Key Concepts

1) The solution to an N-dimensional minimum mean-squared error (MMSE) filtering or least squares problem takes the form

$$\mathbf{w} = \mathbf{R}^{-1}\mathbf{p}$$

where $\mathbf{R} = \mathbf{X}\mathbf{X}^T$ is an N-by-N matrix and the N-by-L matrix $\mathbf{X}$ is composed from the data.

2) Computing the solution using $\mathbf{R}^{-1}$ may be undesirable for several reasons.

a) It requires $L \geq N$ data measurements before the inverse exists.

b) The computational cost of matrix inversion is proportional to N^3 and thus can be excessive if N is large.

c) If L or N is very large, then it may not be possible to store $\mathbf{X}$ in memory.

d) The dynamic range of $\mathbf{R}$ is the square of that in the data $\mathbf{X}$. Thus inverting $\mathbf{R}$ may exacerbate numerical issues when the data has high dynamic range.

3) The least-mean-square (LMS) algorithm offers a low computational cost, iterative approach for finding $\mathbf{w}$ using gradient descent.

a) The current iterate $\mathbf{w}[n]$ is updated with a step in the direction of the negative gradient of the instantaneous squared error $e^2[n]$ where

$$e[n] = d[n] - \mathbf{x}^T[n]\mathbf{w}[n]$$

Here $\mathbf{x}^T[n]$ is the n-th row of $\mathbf{X}^T$.

b) The LMS algorithm is

$$\mathbf{w}[0] = \begin{bmatrix} 0 & 0 & \cdots & 0 \end{bmatrix}^T$$

for $n = 0$ to L

$$e[n] = d[n] - \mathbf{x}^T[n]\mathbf{w}[n]$$
$$\mathbf{w}[n+1] = \mathbf{w}[n] + \mu e[n]\mathbf{x}[n]$$

end loop

where μ is a step size parameter.

4) Features of the LMS algorithm

a) The number of computations at each iteration is proportional to N.

b) The memory requirement at each iteration is also proportional to N.

c) Ideally suited for updating the solution as data is acquired. There is no need to wait until a certain amount of data is acquired and the iteration can continue indefinitely (arbitrarily large L).

d) The instantaneous gradient is not equal to the gradient of the mean squared error $\frac{1}{L}\sum_{n=1}^{L} e^2[n]$.

e) The convergence of the LMS algorithm to an optimum solution depends on μ as discussed in the next lesson.

Least Squares (MMSE Filtering)

$\underline{w}: N \times 1$

$$e[n] = d[n] - \underline{x}^T[n]\,\underline{w} \qquad \min_{\underline{w}} \sum_{n=\ell_1}^{\ell_1+L-1} e^2[n]$$

or

$$\underline{e} = \underline{d} - \underline{X}^T\underline{w} \qquad \min_{\underline{w}} \underline{e}^T\underline{e} \iff \min_{\underline{w}} \|\underline{d} - \underline{X}^T\underline{w}\|_2^2$$

where

$$\underline{e} = \left[e[\ell_1] \cdots e[\ell_1+L-1]\right]^T, \quad \underline{d} = \left[d[\ell_1] \cdots d[\ell_1+L-1]\right]^T$$

$$\underline{X}^T = \begin{bmatrix} \underline{x}^T[\ell_1] \\ \vdots \\ \underline{x}^T[\ell_1+L-1] \end{bmatrix}$$

Time Series:

$$\underline{x}^T[n] = \left[x[n]\; x[n-1] \cdots x[n-N+1]\right]$$

Normal Equations: $\underline{X}\,\underline{X}^T\underline{w} = \underline{X}\underline{d}$ or $\underline{R}\,\underline{w} = \underline{P}$

Direct Solution: $\underline{w} = \underline{R}^{-1}\underline{p}$

- Requires $L \geq N$
- Computational cost of matrix inversion $O(N^3)$
- Update solution as new data is acquired?
- If L, N are very big, may not be able to store $\underline{X}$ in memory
- The dynamic range of $\underline{R}$ is the square of that for $\underline{X}$

Desire low computational cost, iterative solution

Gradient (instantaneous) descent or

Least-Mean-Square (LMS) algorithm

Update current guess (solution) $\underline{w}[n]$ with a step in negative gradient direction of $e^2[n]$

$$\underline{w}[n+1] = \underline{w}[n] - \frac{\mu}{2}\underline{g}[n]$$

Gradient: $\underline{g}[n] = \frac{\partial}{\partial \underline{w}[n]} e^2[n] = \begin{bmatrix} \frac{\partial}{\partial w_0[n]} e^2[n] \\ \vdots \\ \frac{\partial}{\partial w_{N-1}[n]} e^2[n] \end{bmatrix}$

Step size: $\mu/2$

$$\underline{g}[n] = 2\,e[n]\frac{\partial}{\partial \underline{w}[n]} e[n] = 2e[n] \begin{bmatrix} \frac{\partial}{\partial w_0[n]} \\ \vdots \\ \frac{\partial}{\partial w_{N-1}[n]} \end{bmatrix} \left(d[n] - \underline{x}^T[n]\underline{w}[n]\right)$$

But $$\frac{2}{\partial w_\ell[n]}\left(d[n]-\sum_{k=0}^{N-1} w_k[n]\,x[n-k]\right) = -x[n-\ell]$$

So $$\underline{g}[n] = 2e[n]\begin{bmatrix} -x[n] \\ \vdots \\ -x[n-N+1] \end{bmatrix} = -2e[n]\,\underline{x}[n]$$

LMS Algorithm

$$\underline{w}[0] = [0\ 0\ \ldots\ 0]^T \quad \text{(Initialize)}$$

$$e[n] = d[n] - \underline{x}^T[n]\,\underline{w}[n]$$

$$\underline{w}[n+1] = \underline{w}[n] + \mu e[n]\,\underline{x}[n] \quad \text{(Iterate)}$$

$$n = n+1$$

LMS Features

$$e[n] = d[n] - \underline{x}^T[n]\,\underline{w}[n]$$

$$\underline{w}[n+1] = \underline{w}[n] + \mu\, e[n]\,\underline{x}[n]$$

1) Computation cost:
   a) $e[n]$ takes $O(N)$ multiplications
   b) $\underline{w}[n+1]$ takes $O(N)$ multiplications
   vs $O(N^3)$ for $\underline{R}^{-1}\underline{p}$
2) Memory: requires storage of $\underline{x}[n]$, $\underline{w}[n] \rightarrow O(N)$
   vs $O(LN)$ for $\underline{X}, \underline{d}$
3) Ideal for sequential scenarios - update solution as data $(\underline{x}[n], d[n])$ is measured
4) Convergence behavior depends on μ